

PHP et formulaire

PHP et formulaire

- Les formulaires HTML permettent aux utilisateurs de saisir des données.
- PHP récupère et traite ces données côté serveur.
- Utilisation des **superglobales** : \$_GET, \$_POST.

Applications

- Authentification

Se connecter

ou utiliser mon adresse e-mail :

Email

Mot de passe

Je n'ai pas de compte, je m'en crée un.

Se connecter

Formulaire de connexion

Application

- Recherche

Rechercher :

te

test

texte

temps

Envoyer

Application

- Envoi de messages

Votre Mail :

Sujet :

Votre message :

Code anti-spam :

Vous devez reproduire le code de l'image avant d'envoyer le message.

Structure d'un formulaire HTML

- Exemple simple :

```
<form method="POST"
action="traitement.php">
<label>Nom :</label>
<input type="text" name="nom">
<input type="submit" value="Envoyer">
</form>
```

method : définit la façon dont les données sont envoyées (GET ou POST).

action : fichier PHP qui traitera les données.

name : identifiant du champ, utilisé par PHP pour récupérer la valeur.

Formulaire: method

Les méthodes principales

Il existe deux méthodes :

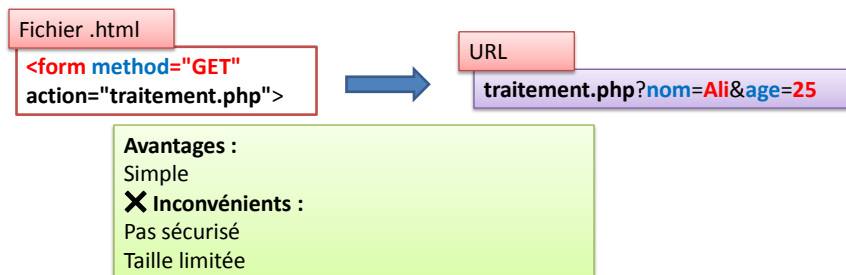
? **GET**

? **POST**

Formulaire: method GET

Caractéristiques :

- Les données sont visibles dans l'URL
- Exemple :



Formulaire: method POST

Caractéristiques :

Données envoyées dans le corps de la requête
(non visibles)
Plus sécurisé que GET

Fichier .html

```
<form method="POST"
action="traitement.php">
```

URL

données **cachées** ✗ (pas dans l'URL)

Avantages :

Pas de limite importante de taille
Utilisé pour données sensibles (mot de passe)

✗ Inconvénients :

Non visible dans l'URL alors Non bookmarkable (mettre en favori)
Impossible de Partager le lien

Récupération en PHP

• Avec **GET** :

```
$nom = $_GET['nom'];
```

Avec **POST** :

```
$nom = $_POST['nom'];
```

Bonnes pratiques

Utiliser **POST** pour : **mot de passe, données sensibles**
Utiliser **GET** pour : **recherche, filtres**

Récupération des données en PHP

- Exemple avec POST :

```
<?php
$nom = $_POST['nom'];
echo "Bonjour, " . $nom;
?>
```

`$_POST['nom']` contient la valeur saisie par l'utilisateur.
Affichage dynamique en fonction de l'entrée.

L'accès au champs d'un formulaire

Les boutons radio

Fruits :

```
<input type="radio" name="fruit" value="Banane"> Banane
<input type="radio" name="fruit" value="Pomme"> Pomme
```

name= identifie le groupe de boutons
Value: valeur envoyer au serveur

```
$fruits = $_POST['fruit'];
```

Les cases à cocher (checkbox)

Langages :

```
<input type="checkbox" name="lang[]" value="PHP"> PHP
<input type="checkbox" name="lang[]" value="JS"> JS
```

```
foreach($_POST['lang'] as $l){
    echo $l;
}
```

L'accès au champs d'un formulaire

- Liste déroulante (select)

```
<select name="ville">
  <option value="Tunis">Tunis</option>
  <option value="Sfax">Sfax</option>
</select>
```

```
$ville = $_POST['ville'];
```

L'accès au champs d'un formulaire

- Upload fichier:

Formulaire HTML pour upload

```
<form action="upload.php" method="POST" enctype="multipart/form-data">
  <input type="file" name="fichier">
  <input type="submit" value="Envoyer">
</form>
```

type="file" → permet de choisir un fichier **enctype** → obligatoire pour envoyer des fichiers
L'attribut enctype sert à définir **comment les données sont encodées (formatées) avant d'être envoyées au serveur.**

Oui → sans multipart les données sont **modifiées (encodées)**

✓ Avec multipart → les données sont **envoyées telles quelles (surtout les fichiers)**

En PHP, on utilise la variable globale : **\$_FILES**

L'accès au champs d'un formulaire

Récupération du fichier en PHP

```
$_FILES['fichier']['name']; // nom du fichier
$_FILES['fichier']['type']; // type MIME
$_FILES['fichier']['tmp_name']; // chemin temporaire
PHP ne le met pas directement dans le dossier Il le
stocke temporairement sur le serveur Ce fichier est
supprimé automatiquement après le script .
$_FILES['fichier']['error']; // code erreur: C'est un
code qui indique si l'upload s'est bien passé. (0: aucun
problème)
$_FILES['fichier']['size']; // taille en octets
```

Structure de \$_FILES

```
$_FILES['fichier'] = [
    'name' => 'image.jpg', // nom du
    fichier
    'type' => 'image/jpeg', // type MIME
    'tmp_name' => '/tmp/xyz', // fichier
    temporaire
    'error' => 0, // code erreur
    'size' => 12345 // taille en octets
];
```

L'accès au champs d'un formulaire

- Déplacer le fichier vers un dossier:

```
move_uploaded_file($_FILES['fichier']['tmp_name'], "uploads/" .
$_FILES['fichier']['name']);
```

Exemple complet :

```
if (isset($_FILES['fichier'])) {
    $nom = $_FILES['fichier']['name'];
    $tmp = $_FILES['fichier']['tmp_name'];

    move_uploaded_file($tmp, "uploads/" . $nom);
    echo "Fichier uploadé avec succès";
}
```

Sécurisation (Fichier)

- Vérifier **les erreurs**

```
if ($_FILES['fichier']['error'] == 0) {
    echo "Pas d'erreur";
}
```

- Limiter **la taille**

```
if ($_FILES['fichier']['size'] < 1000000) { // < 1 Mo
```

- Vérifier **l'extension**

```
$extension = pathinfo($_FILES['fichier']['name'], PATHINFO_EXTENSION);
if (in_array($extension, ['jpg', 'png', 'pdf'])) {
```

Extraction de l'extension: `pathinfo($_FILES['fichier']['name'], PATHINFO_EXTENSION)`;
Exemple: `document.pdf` retourne: `pdf`

Sécurisation (Fichier)

- Créer un nouveau nom unique pour un fichier uploadé: (**éviter les conflits**):

```
$nouveauNom = uniqid() . "." . $extension;
```

- téléchargement d'un fichier

```
//On définit le chemin du fichier sur le serveur
$fichier = "uploads/test.pdf";
// Indique au navigateur :le type du fichier est un PDF
//donc il doit être traité comme un document PDF
header("Content-Type: application/pdf");
// force le téléchargement X pas affichage direct filename=test.pdf
//nom du fichier téléchargé côté utilisateur
header("Content-Disposition: attachment; filename=test.pdf");
//lit le fichier sur le serveur, envoie son contenu au navigateur et
//déclenche le téléchargement
readfile($fichier);
```

Vérification des données

Vérification des données (isset)

- **isset()**
- **Empty()**

Vérification des données (isset)

- **isset()** est une fonction PHP Elle permet de vérifier si une variable : **existe**
- et **n'est pas NULL**

Vérification des données (isset)

- Vérifier l'existence des données:

```
if (isset($_POST['nom'])) {  
    echo $_POST['nom'];  
}
```

Retourne :

✓ true → si la variable existe et ≠ NULL

✗ false → si :

la variable n'existe pas
ou elle vaut NULL

Vérification des données (empty)

- Vérifie si le champ est vide

```
if (empty($_POST['nom'])) {  
    echo "Champ vide";  
}
```

En PHP, empty() considère comme "vide" plusieurs valeurs :

- ✓ 0
- ✓ "0"
- ✓ ""
- ✓ null
- ✓ false
- ✓ tableau vide []

Sécurisation des données(htmlspecialchars())

- Éviter les attaques (XSS, injection)

```
$nom = htmlspecialchars($_POST['nom']);
```

htmlspecialchars() transforme du code HTML/JS en simple texte (chaîne de caractères)

Nettoyage et sécurité (trim())

Enlève espaces

```
$nom = htmlspecialchars(trim($_POST['nom']));
```

Nettoyage et sécurité (filter_var)

- ✓ Valider les données utilisateur
- ✓ Nettoyer les entrées (sécurité)
- ✓ Éviter les erreurs et injections
- ✓ Garantir des données correctes

Syntaxe:

```
filter_var(valeur, filtre, options);
```

valeur : donnée à traiter

filtre : type de validation/nettoyage

options : paramètres supplémentaires

Nettoyage et sécurité (filter_var)

• Validation email

```
$email = "test@gmail.com";

if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email valide";
} else {
    echo "Email invalide";
}
```

FILTER_VALIDATE_EMAIL vérifie :

- ✓ format correct
- ✓ présence de @
- ✓ domaine valide
- ✓ extension

=> Vérifie si l'email est correct

• Validation d'une URL

```
$url = "https://example.com";

if (filter_var($url, FILTER_VALIDATE_URL)) {
    echo "URL valide";
}
```

FILTER_VALIDATE_URL vérifie :

- ✓ protocole (http, https...)
- ✓ structure correcte
- ✓ domaine valide

Nettoyage et sécurité (filter_var)

• intervalle d'un entier

```
$age = 20;

$options = [
    "options" => [
        "min_range" => 18,
        "max_range" => 30
    ]
];

if (filter_var($age, FILTER_VALIDATE_INT, $options)) {
    echo "Age valide";
}
```

- ✓ Retourne la valeur → si valide
- ✗ Retourne false → si invalide